

EXP 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:
Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. Target Hash Generation with Nonce:
Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. Proof-of-Work Puzzle Solving:
Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. Merkle Tree Construction:
Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

... Enter a string: mansi is executing exp in lab 507
Hash: 2dae6c08de59b5496a79ccb6fd1d599b14706fc72ecabbd37d644c65662002f3

```

▶ import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)

```

... Enter a string: mansi is executing exp in lab 507
 Enter the nonce: 13
 Hash Code: 9878841d8aeee7496600673a9cfaa8ccb6ef3cbf4cf2dbf1d483e447679a1f68

```

▶ import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

        nonce += 1

# Get user input
input_string = "Mansi is executing exp in lab 507;"
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

... Hash: 0692a6bde3a9b605055c2810071a28efc782ca83d6ede471f9c9613d6ff1717a
 Input String: Mansi is executing exp in lab 507;
 Leading Zeros: 1
 Expected Nonce: 25

```

import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    if len(transactions) == 1:
        return transactions

    tree = [transactions] # Store initial leaf nodes (Level 0)

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0: # Check if odd number of transactions
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i + 1]
            hash_combined = hashlib.sha256(
                combined.encode("utf-8")
            ).hexdigest()
            new_transactions.append(hash_combined)

        transactions = new_transactions
        tree.append(
            transactions
        ) # Add the newly created hash level to the tree

    return tree

```

```

# Example usage
transactions = [
    "mansi to yash 4 BTC", "luffy to zoro 6 BTC", "Doffy to Nami 10 BTC", "Robin to Chopper 1 BTC", "Jimbei to sanji 3 BTC",
]
merkle_tree = build_merkle_tree(transactions)

# Print the complete tree level by level
print("Entire Merkle Tree:")
for level_idx, level in enumerate(merkle_tree):
    print(f"Level {level_idx}: {level}")

```

```

... Entire Merkle Tree:
Level 0: ['mansi to yash 4 BTC', 'luffy to zoro 6 BTC', 'Doffy to Nami 10 BTC', 'Robin to Chopper 1 BTC', 'Jimbei to sanji 3 BTC', 'Jimbei to sanji 3 BTC']
Level 1: ['f964e5a6abcf54ac31445590afb19f3c73a70e6a5dd94ee1d98aecf737248d1c', '15d66aa0211e699f8696932def0b1052d79b99ad5ac4cbcd732c15f42b810ca0', '65536c1d']
Level 2: ['fa7a41162caca1568e7adeafafa13a105ddb7bc16491d639b5bb05d5dd5ae9f27', '5089c239faa2f9090aeeb3cd2df2b36e84550f9946a050a901aed28f531d4e05']
Level 3: ['d00d55777a542ece5bc2250d7124e57eaa490b924f8e9b04e1ddd63c5ba65134']

```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- SHA-256 Hashing provides a secure and unique fingerprint for any input, ensuring data integrity.
- Nonce and Target Hashing allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- Proof-of-Work Puzzle Solving shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- Merkle Tree Construction efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.